

Setup do Challenge – Requisitos de Ambiente

Este documento apresenta os requisitos técnicos mínimos para participação nos Challenges do Future Minds 3.

Recomenda-se que todos os participantes validem previamente seus acessos, ferramentas e configurações em conjunto com o seu grupo, garantindo que o ambiente esteja pronto antes do início da atividade. O objetivo é permitir que as 4 horas de challenge sejam totalmente dedicadas à construção da solução, evitando perdas de tempo com setup ou ajustes técnicos.

Os grupos foram formados de maneira mista, com participantes utilizando máquinas Zup e Itaú. Nesse contexto, é importante considerar que algumas ferramentas e serviços (como AWS Bedrock, GitHub Copilot, entre outros) podem ter restrições com a máquina Zup, devido a acesso e licenças específicas.

Dessa forma, cada grupo deverá se organizar previamente para garantir que possui, de forma conjunta, acesso às ferramentas necessárias para execução do desafio, podendo optar por diferentes abordagens e plataformas disponíveis.

Para assegurar privacidade e segurança da informação, todos os participantes deverão trabalhar exclusivamente com dados sintéticos durante o desafio. Não é permitido o uso de dados reais, sensíveis ou produtivos em nenhuma etapa da atividade.

Além disso, recomenda-se seguir as orientações de uso das plataformas e ferramentas indicadas, respeitando as boas práticas de configuração e utilização de recursos. Essas diretrizes têm como objetivo garantir uma experiência fluida, segura e equivalente para todos os grupos.

Instruções: Caso tenha algum impedimento nesta validação, entre em contato com o time Autoridade Tech (autoridadetech@zup.com.br)

Para execução do desafio, valide os seguintes pré-requisitos:

1. Python e Ambiente (a partir do nível BIT)

- Recomendado Python 3.10 ou superior instalado → `python --version`
- pip atualizado → `pip install --upgrade pip`
- Ambiente virtual criado (venv ou conda) → `python -m venv finguard_env`

1.1 Bibliotecas de exemplo - SDK e Integração AWS

Python

```
pip install boto3 langchain langchain-aws langchain-community langgraph faiss-cpu
```

1.2 Bibliotecas de exemplo - Dados e Visualização

Python

```
pip install pandas plotly matplotlib scikit-learn
```

2. AWS - Credenciais e acesso (a partir do nível BIT)

2.1 Conta e Credenciais

Verificar se já possui acesso à conta AWS no seu projeto:

- Acesso à https://<nome_da_conta>.awsapps.com/start/#/?tab=accounts
- Contas > Chaves de acesso > Opção 1: definição de variáveis de ambiente da AWS
- Se o console AWS estiver disponível para uso:
- AWS CLI instalado → `aws --version`
- Credenciais configuradas → `aws configure` (Access Key, Secret Key, Region)
- Região correta definida (validação com o organizador do desafio)
- Identidade confirmada → `aws sts get-caller-identity`

Tipos de conta que poderão ser usadas:

- Conta DevOps (desenvolvimento)
Conta Consumer

Caso não tenha acesso à uma conta AWS para utilização do Bedrock:

Siga a documentação do AWS Bedrock para jornada em conta devops e conta consumer respectivamente:

<https://github-pages-dev.cloud.itaú.com.br/itaú-kk7-doc-bedrock/docs/documentacao/features/bedrock-guardrail/devops/guia/>

<https://github-pages-dev.cloud.itaú.com.br/itaú-kk7-doc-bedrock/docs/documentacao/features/bedrock-guardrail/consumer/guia/>

2.2 Acesso ao Console

- Console do Bedrock acessível → <https://console.aws.amazon.com/bedrock/>
- Modelos habilitados em "Model access": Amazon Nova Lite (amazon.nova-lite-v1:0)
- Confira previamente se os recursos do Bedrock estão disponíveis. Pode haver limitações conforme o projeto que está alocado.

3. Teste de Chamada ao Modelo (Nova Lite) - Exemplo para validação (a partir do nível BIT)

Python

```
import boto3
import json
bedrock = boto3.client(service_name="bedrock-runtime", region_name="us-west-2")
response = bedrock.invoke_model(
    modelId="us.amazon.nova-lite-v1:0",
    body=json.dumps({
        "messages": [
            {"role": "user", "content": [{"text": "Diga 'teste OK'"}]}
        ],
        "inferenceConfig": {"temperature": 0, "max_new_tokens": 50}
    }),
    contentType="application/json",
    accept="application/json"
)

result = json.loads(response["body"].read())
print(result["output"]["message"]["content"][0]["text"])
```

- O modelo Nova Lite respondeu adequadamente?

3.1 Teste RAG Local (FAISS + Titan Embeddings)

Python

```
from langchain_aws import BedrockEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document

embeddings = BedrockEmbeddings(
    model_id="amazon.titan-embed-text-v2:0",
    region_name="us-east-1"
)

docs = [
    Document(page_content="Urgência CRÍTICA: prazo de resposta até 4 horas."),
    Document(page_content="Cartão de crédito: estorno provisório em até 48h."),
]

vectorstore = FAISS.from_documents(docs, embeddings)
results = vectorstore.similarity_search("prazo para fraude no cartão", k=1)
print(f"Resultado RAG: {results[0].page_content}")
```

- FAISS criou o vectorstore local?
- A busca semântica retornou um resultado relevante?

Caso não consiga realizar essa validação, o desafio poderá ser executado localmente.

4. AWS - SageMaker (a partir do nível TERABYTE - EXTRA)

4.1 Acesso ao Console

- Console do SageMaker acessível → <https://console.aws.amazon.com/sagemaker/>
- JupyterLab acessível (Studio ou Notebook Instance)
- Amazon Q disponível no JupyterLab

4.2 Permissões SageMaker

Python

```
import boto3

sm = boto3.client("sagemaker")
print(sm.list_training_jobs(MaxResults=1))
```

- Permissão para criar Training Jobs
- Permissão para criar Endpoints
- Permissão para acessar S3

4.3 Acesso ao S3

Python

```
s3 = boto3.client("s3")
buckets = s3.list_buckets()
print([b["Name"] for b in buckets["Buckets"]])
```

- A listagem de buckets foi bem-sucedida
- Bucket disponível para armazenamento de dados/modelo

5. Microsoft Copilot Studio (Alternativa para níveis BIT, BYTE E TERABYTE)

5.1 Acesso ao Ambiente

- Copilot Studio acessível → <https://copilotstudio.microsoft.com/>
- Login com conta corporativa (Itaú - @mailer) efetuado com sucesso
- Checar permissão para criar novos agentes/copilots

Documentação do Copilot Studio no Itaú:

<https://iconectados.sharepoint.com/sites/CopilotStudio>

5.2 Funcionalidades

- Criação de Agente (Copilot Agent) → Testar criação e exclusão
- Criação de Tópicos (equivalente a guardrails) → Testar criação de regra simples
- Knowledge Source (equivalente a RAG) → Testar upload de documento
- Orquestração de fluxo → Testar criação de fluxo com 2 ou mais etapas

6. Itaú - Iara GenAI SDK (se aplicável)

Seguir as diretrizes no portal:

<https://github-pages-dev.cloud.itaubr.com.br/itau-kk7-doc-iara-gen-ai/docs/documentacao/com-ece-por-aqui/pre-requisitos/>

- 1 - É necessária a sigla APP para a geração de **client_id** e **client_secret** (deve ser criado o repositório da aplicação, e cadastrada no **defectDOJO** por um arquiteto).
 - 2 - Geração da credencial no portal de credenciais.
 - 3 - Autorização dos escopos no portal de credenciais.
- SDK Iara instalado → `pip install <pacote_iara>`
 - Credenciais Iara configuradas
 - Teste de chamada ao modelo via Iara SDK
 - Documentação do SDK acessada e disponível



7.Assistentes de Código

7.1 GitHub Copilot

- Extensão instalada no VS Code (ou IDE de preferência)
- Licença ativa e em pleno funcionamento
- Teste: Abrir arquivo .py e validar as sugestões de código

Documentação do Github Copilot no Itaú:

<https://iconectados.sharepoint.com/sites/SquadDevTools/SitePages/Git-Hub-Copilot.aspx>

7.2 Amazon Q Developer

- Extensão instalada no VS Code ou disponível no SageMaker JupyterLab
- Teste: Solicitar sugestão de código no notebook